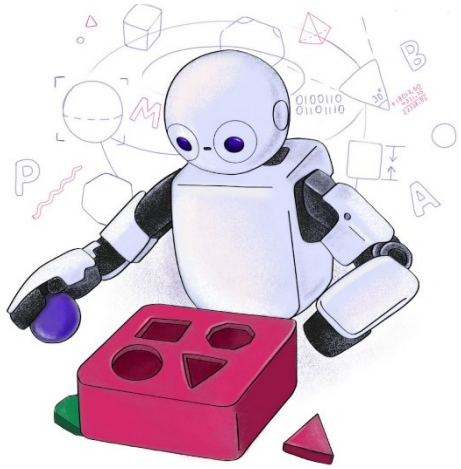
TP558 - Tópicos avançados em Machine Learning:

Generative Adversarial Nets (GAN)



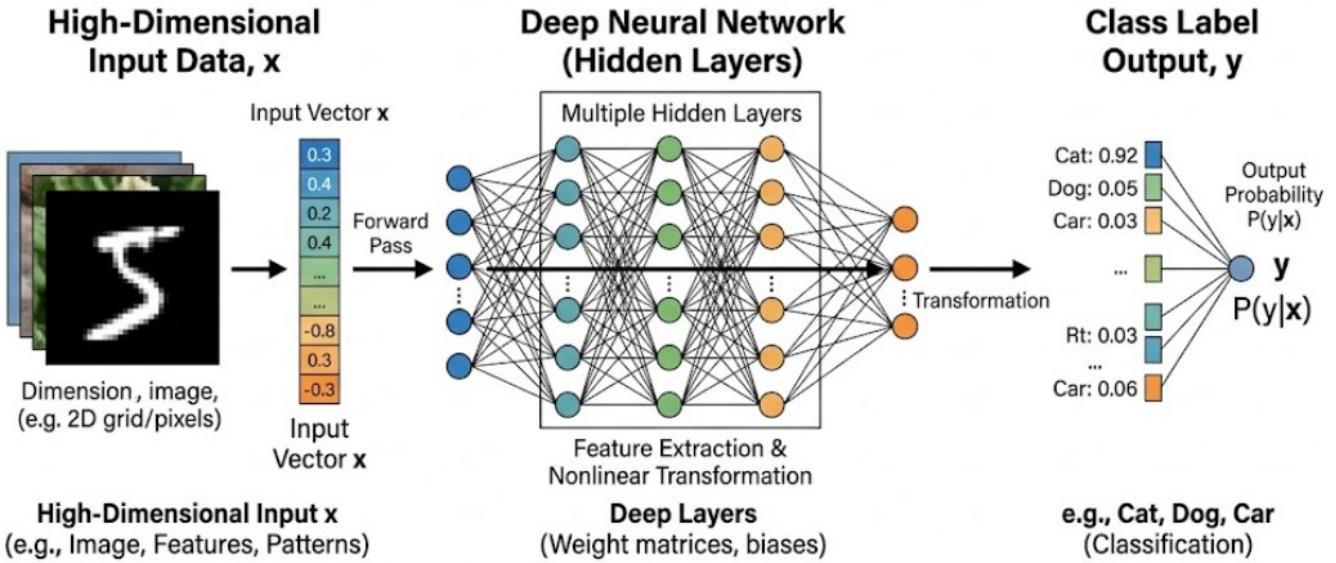
Inatel

Bryan Fernando Sarango Rodriguez
bryan.rodriguez@dtel.Inatel.br

Generative Adversarial Nets

Ian J. Goodfellow, Jean Pouget-Abadie*, Mehdi Mirza, Bing Xu, David Warde-Farley,
Sherjil Ozair†, Aaron Courville, Yoshua Bengio‡
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

Contexto (2014)

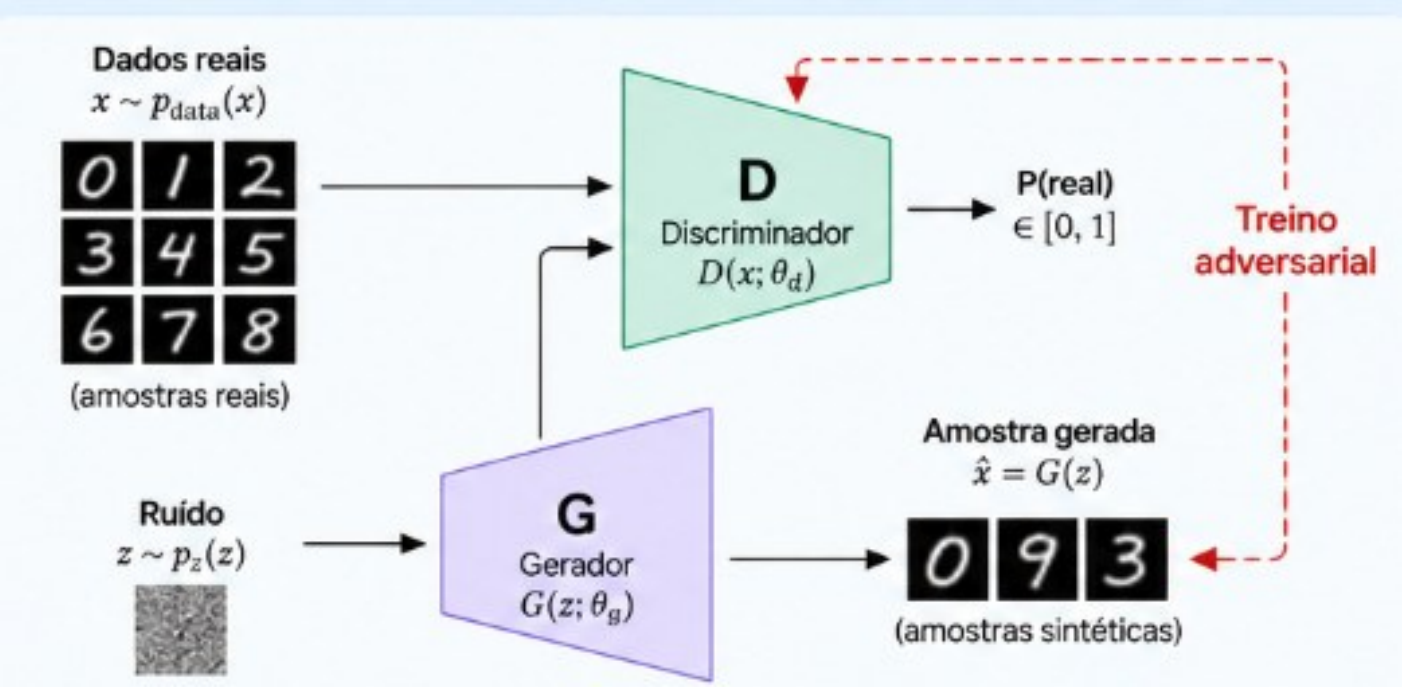


- Sucesso do deep learning: modelos discriminativos (x → rótulo).
- Apoiado em backprop, dropout e unidades lineares por partes.
- Modelos generativos ficavam para trás: verossimilhança intratável e difícil de combinar com essas técnicas.

Pergunta de pesquisa

É possível treinar um modelo só com backpropagation, sem cadeias de Markov Markov nem inferência aproximada?

Contribuições do artigo



Arcabouço adversarial

G gera e D discrimina, treinados juntos em um jogo minimax.

Ótimo global com prova

Solução única: $p_g = p_{\text{data}}$ e D = 1/2, via Jensen–Shannon.

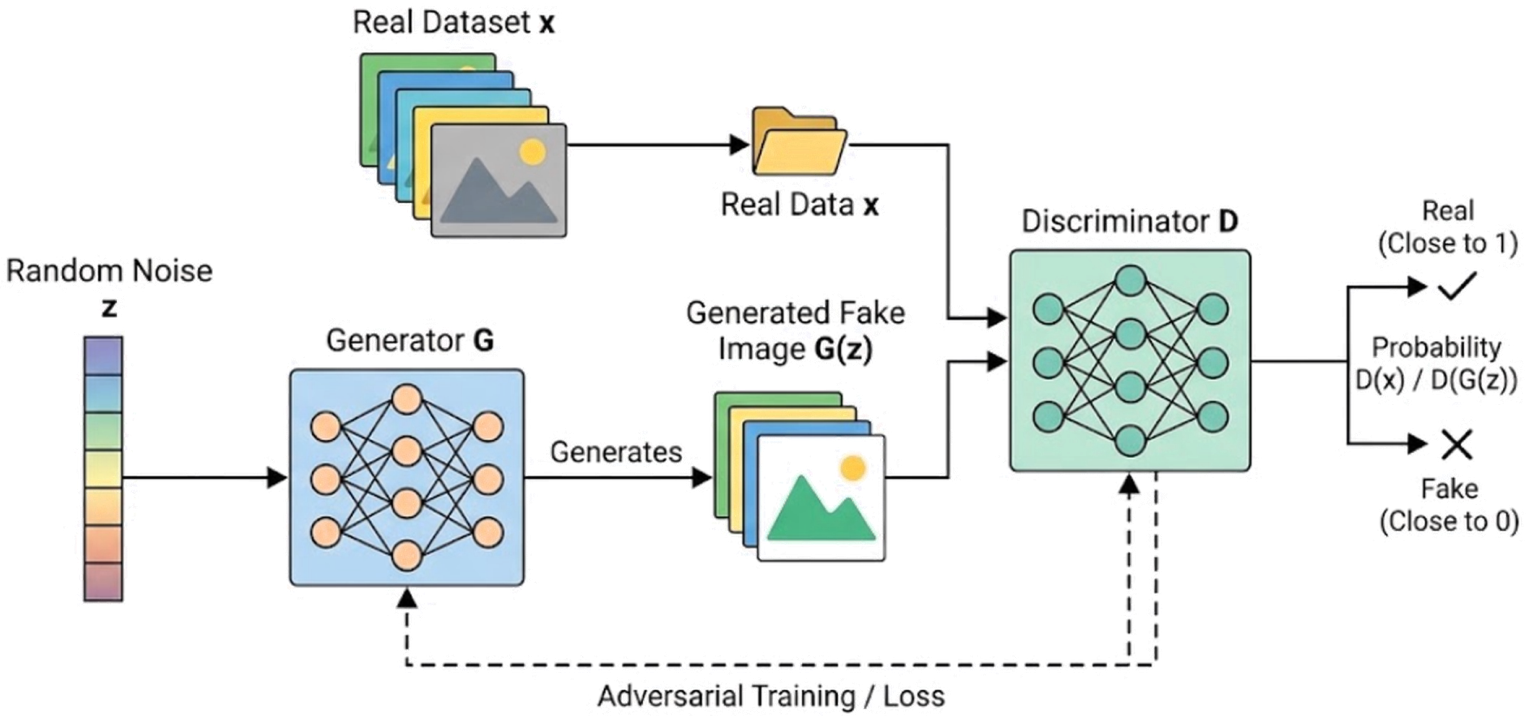
Treino só com backprop

Sem cadeias de Markov nem inferência aproximada.

Validação empírica

Amostras competitivas em MNIST, TFD e CIFAR-10, ideia funciona.

Gerador e discriminador: arquitetura



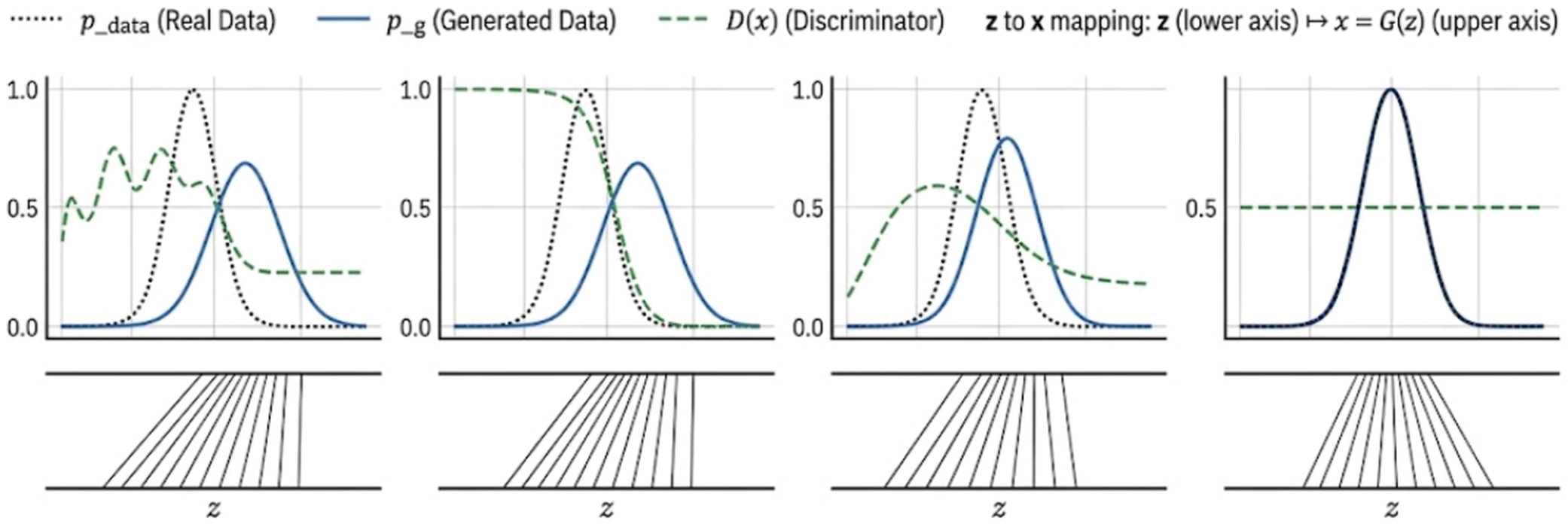
Gerador G

Rede diferenciável MLP que transforma ruído em amostras. Ativações ReLU e sigmoide; ruído só na camada de entrada. Nunca vê dados reais: aprende apenas pelo gradiente que atravessa D.

Discriminador D

MLP classificador binário: recebe amostras reais e geradas e devolve a probabilidade de serem reais. Ativações maxout, treinado com dropout.

Intuição: como as distribuições evoluem



(a) Perto da convergência: p_g parecida com p_data; D é um classificador apenas parcialmente preciso.

(b) Laço interno: D é treinado e converge para

$$D^*(x) = \frac{p_{data}}{(p_{data} + p_g)}.$$

(c) Após atualizar G, o gradiente de D empurra G(z) para regiões mais classificadas como dados.

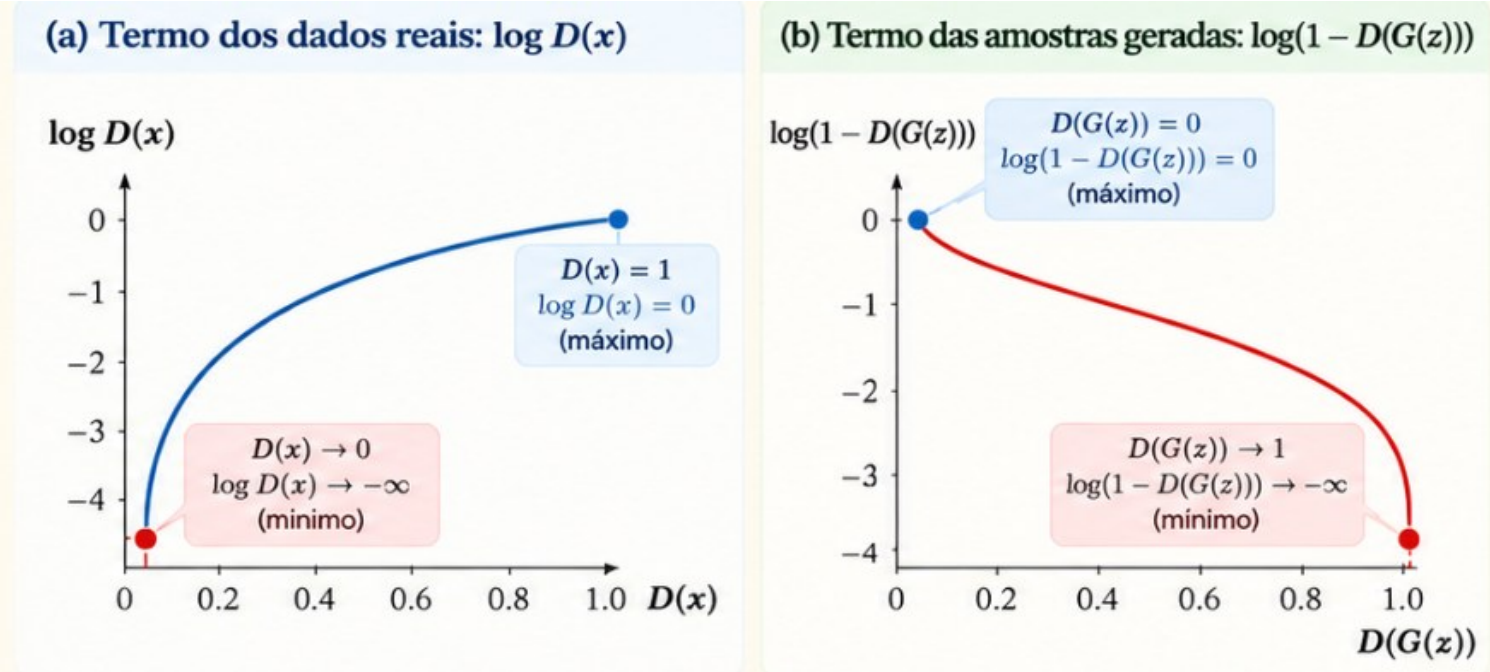
(d) Equilíbrio: p_g = p_data e D(x) = 1/2. D não consegue mais distinguir.

A função de perda minimax

$$\min_G \max_D \; V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}\big[\log D(\boldsymbol{x})\big] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}\big[\log\big(1 - D(G(\boldsymbol{z}))\big)\big]$$

- $p_{\text{data}}(x)$ é a distribuição de probabilidade dos dados reais de treinamento e p(z) é a distribuição do vetor de ruído z. Normalmente, p(z) é gaussiana
- O gerador tenta **minimizar** esta função, enquanto o discriminador tenta **maximizá-la**.
- D(x) e D(G(z)) são valores de probabilidade: ambos estão no intervalo entre 0 e 1.
- 1º termo: recompensa D por reconhecer amostras reais.
- 2º termo: recompensa D por rejeitar amostras geradas, e é o único termo que G controla.

Função monotônica: por que o logaritmo



- log é uma função monotônica: se *f(y)* é monotonicamente crescente, à medida que *y* aumenta, *f(y)* também aumenta ou permanece constante.
- Assim, log D(x) é maximizado quando D(x) = 1 e minimizado quando D(x) = 0.
- Da mesma forma, log(1 – D(G(z))) é maximizado quando D(G(z)) = 0 e minimizado quando D(G(z)) = 1.

Função de perda do discriminador

$$\max_{D} \; \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}\Big[\log D(\boldsymbol{x})\Big] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}}\Big[\log\Big(1 - D(G(\boldsymbol{z}))\Big)\Big]$$

- Do ponto de vista de D, o problema é só uma classificação binária supervisionada: dizer se uma imagem é real ou falsa.
- Queremos que o discriminador preveja todos os D(x) como 1 e todos os D(G(z)) como 0.
- Portanto, o discriminador irá maximizar D(x) e minimizar D(G(z)) para maximizar globalmente a função de perda acima.
- Durante este passo, G fica congelado: só os parâmetros θ_d são atualizados.

> Equivalência: maximizar essa expressão é minimizar a entropia cruzada binária de um classificador real/falsa.
>
> $$\mathcal{L}_{\text{BCE}}(y,\hat{y}) = -\big[\, y\log\hat{y} + (1-y)\log(1-\hat{y})\,\big], \qquad y=1\ (\text{real}),\quad y=0\ (\text{falsa})$$

Função de perda do gerador

$$\min_G \underbrace{\mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}\left[\log D(\boldsymbol{x})\right]}_{\text{não depende de } G} + \mathbb{E}_{\boldsymbol{z}\sim p_z}\left[\log\left(1 - D(G(\boldsymbol{z}))\right)\right]$$

$$\Longrightarrow \quad \min_G \mathbb{E}_{\boldsymbol{z}\sim p_z}\left[\log\left(1 - D(G(\boldsymbol{z}))\right)\right]$$

- Queremos que o gerador G gere amostras que enganem o discriminador, de forma que D(G(z)) = 1.
- Portanto, o gerador deve maximizar D(G(z)) para minimizar globalmente a função de perda acima.
- O gerador não tem controle sobre o primeiro termo; portanto, minimizará apenas o segundo termo da equação.
- O primeiro termo só aparece acima para que a função seja expressa como o mesmo V(D, G) da formulação minimax.

Na prática: a perda não saturante do gerador

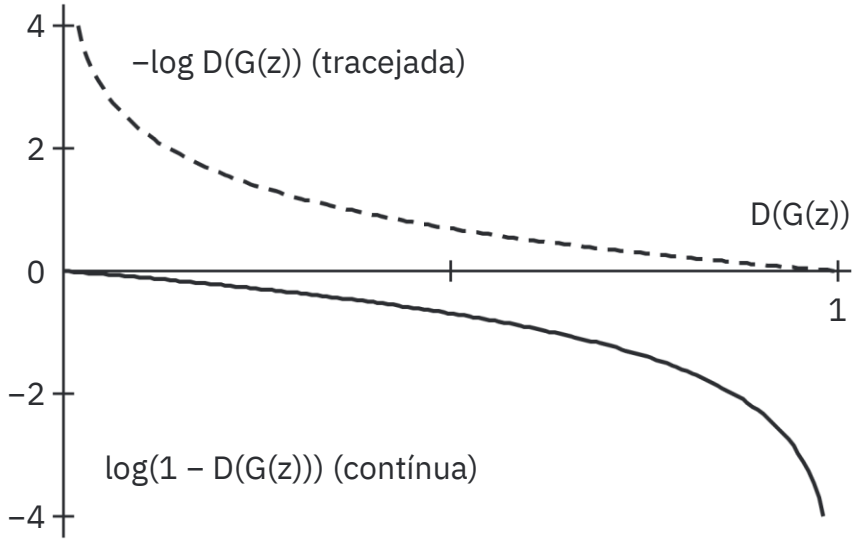
$$\min_G \ \mathbb{E}_{\boldsymbol{z}}\big[\log(1 - D(G(\boldsymbol{z})))\big] \quad \longrightarrow \quad \max_G \ \mathbb{E}_{\boldsymbol{z}}\big[\log D(G(\boldsymbol{z}))\big]$$

$$D = \sigma(a) \qquad (a = \text{logit de } D)$$

$$\frac{\partial}{\partial a} \log\big(1 - \sigma(a)\big) = -\sigma(a) \xrightarrow{D \to 0} 0 \quad \text{(satura)}$$

$$\frac{\partial}{\partial a} \log \sigma(a) = 1 - \sigma(a) \xrightarrow{D \to 0} 1 \quad \text{(gradiente forte)}$$

No início, G é ruim e D rejeita suas amostras com alta confiança: D(G(z)) ≈ 0. Aí log(1 – D(G(z))) satura e o gradiente some. Maximizar log D(G(z)) leva ao mesmo ponto fixo da dinâmica, mas com gradientes muito mais fortes no início.



Ambas curvas são perdas que G minimiza

Discriminador ótimo (Proposição 1)

Proposição 1. Para G fixo, o discriminador ótimo é

$$D_G^*(\boldsymbol{x}) = \frac{p_{\text{data}}(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x}) + p_g(\boldsymbol{x})}$$

Prova. Escrevemos V como integral em x, usando a mudança de variável x = G(z) (z ~ p_z induz x ~ p_g):

$$V(G,D) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}\big[\log D(\boldsymbol{x})\big] + \mathbb{E}_{\boldsymbol{z}\sim p_z(\boldsymbol{z})}\big[\log\big(1 - D(G(\boldsymbol{z}))\big)\big]$$

$$V(G,D) = \int_{\boldsymbol{x}} p_{\text{data}}(\boldsymbol{x}) \log D(\boldsymbol{x})\, d\boldsymbol{x} + \int_{\boldsymbol{z}} p_{\boldsymbol{z}}(\boldsymbol{z}) \log\big(1 - D(G(\boldsymbol{z}))\big)\, d\boldsymbol{z}$$

$$= \int_{\boldsymbol{x}} \Big[p_{\text{data}}(\boldsymbol{x}) \log D(\boldsymbol{x}) + p_g(\boldsymbol{x}) \log\big(1 - D(\boldsymbol{x})\big) \Big]\, d\boldsymbol{x}$$

Maximizamos ponto a ponto em x, com a = p_data(x) e b = p_g(x), para todo (a, b) ≠ (0, 0):

$$f(y) = a\log y + b\log(1-y), \qquad f'(y) = \frac{a}{y} - \frac{b}{1-y} = 0 \;\Rightarrow\; y^* = \frac{a}{a+b}, \qquad f''(y) < 0$$

O critério virtual de treinamento C(G)

C(G) é o critério que estaría minimizando se D fosse sempre ótimo.
Substituindo D*_G em V, o jogo minimax passa a depender só de G :

$$
\begin{aligned}
C(G) &= \max_D V(G, D) \\
&= \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}} \left[\log D_G^*(\boldsymbol{x}) \right] + \mathbb{E}_{\boldsymbol{z} \sim p_z} \left[\log \left(1 - D_G^*(G(\boldsymbol{z})) \right) \right] \\
&= \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}} \left[\log D_G^*(\boldsymbol{x}) \right] + \mathbb{E}_{\boldsymbol{x} \sim p_g} \left[\log \left(1 - D_G^*(\boldsymbol{x}) \right) \right] \\
&= \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}} \left[\log \frac{p_{\text{data}}(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x}) + p_g(\boldsymbol{x})} \right] + \mathbb{E}_{\boldsymbol{x} \sim p_g} \left[\log \frac{p_g(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x}) + p_g(\boldsymbol{x})} \right]
\end{aligned}
$$

- Da 2ª para a 3ª linha: a mesma mudança de variável, x = G(z) com z ~ p(z) equivale a x ~ p_g.
- Na 4ª substituímos a fórmula de D*_G = p_g / (p_data + p_g). Agora minimizar C(G) é o problema de G.

Teorema 1: ótimo global e Jensen–Shannon

1 · Valor no ótimo

$$p_g = p_{\text{data}} \;\Rightarrow\; D_G^*(\boldsymbol{x}) = \tfrac{1}{2} \;\Rightarrow\; C(G) = \log \tfrac{1}{2} + \log \tfrac{1}{2} = -\log 4$$

2 · Somar e subtrair log 2

$$\log \frac{p_{\text{data}}}{p_{\text{data}} + p_g} = \log \frac{p_{\text{data}}}{\frac{1}{2}(p_{\text{data}} + p_g)} - \log 2 \qquad (\text{idem para } p_g)$$

3 · Reconhecer duas KL

$$C(G) = -\log 4 + \mathrm{KL}\left(p_{\text{data}} \,\middle\|\, \frac{p_{\text{data}} + p_g}{2}\right) + \mathrm{KL}\left(p_g \,\middle\|\, \frac{p_{\text{data}} + p_g}{2}\right)$$

4 · Definições

$$\mathrm{KL}(p \,\|\, q) = \int p(\boldsymbol{x}) \log \frac{p(\boldsymbol{x})}{q(\boldsymbol{x})}\, d\boldsymbol{x} \geq 0, \qquad \mathrm{JSD}(p \,\|\, q) = \tfrac{1}{2}\mathrm{KL}(p \,\|\, m) + \tfrac{1}{2}\mathrm{KL}(q \,\|\, m), \;\; m = \tfrac{p+q}{2}$$

5 · Resultado

$$C(G) = -\log 4 + 2 \cdot \mathrm{JSD}\left(p_{\text{data}} \,\|\, p_g\right) \;\geq\; -\log 4$$

> Como JSD ≥ 0, e só vale 0 quando as distribuições são iguais, C* = −log 4 é o mínimo global de C(G), atingido se e somente se p_g = p_data.

Algoritmo 1: SGD em minibatch

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

  for k steps **do**

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
  - Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

  end for

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Detalhes

- k é um hiperparâmetro; os autores usaram k = 1, a opção mais barata.
- As médias no minibatch são estimativas estocásticas das esperanças de V.
- Qualquer regra de gradiente serve; usaram momentum.
- Na prática, G pode subir o gradiente de log D(G(z)) (perda não saturante).

Experimento

$$\hat{p}_g(\boldsymbol{x}) = \frac{1}{N}\sum_{j=1}^{N}\mathcal{N}\big(\boldsymbol{x};\, G(\boldsymbol{z}_j),\, \sigma^2 I\big), \qquad \mathrm{LL} = \frac{1}{|\mathcal{T}|}\sum_{\boldsymbol{x}\in\mathcal{T}}\log \hat{p}_g(\boldsymbol{x})$$

Dados

MNIST (dígitos), Toronto Face Database (TFD) e CIFAR-10 (imagens naturais).

Métrica

Janela de Parzen gaussiana ajustada às amostras de G; σ escolhido por validação cruzada. Alta variância e fraca em alta dimensão.

Baselines

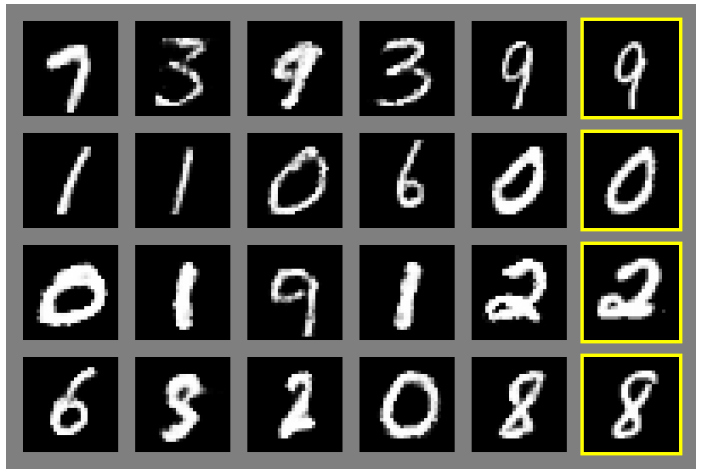
DBN, Stacked CAE e Deep GSN.

Modelo	MNIST	TFD
DBN	138 ± 2	1909 ± 66
Stacked CAE	121 ± 1,6	2110 ± 50
Deep GSN	214 ± 1,1	1890 ± 29
Adversarial nets	225 ± 2	2057 ± 26

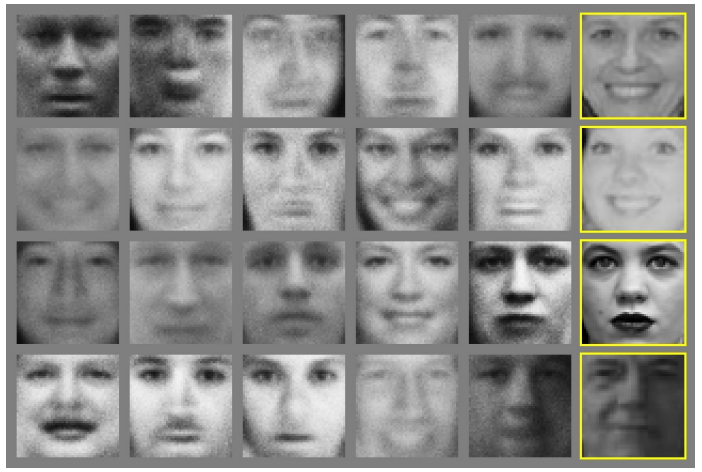
Tabela 1: log-verossimilhança estimada por Parzen (maior é melhor).

> **Leitura:** a GAN é a melhor no MNIST (225) e a segunda no TFD (2057), atrás do Stacked CAE (2110).

Resultados visuais:



a) MNIST



b) TFD



Fig. 1: a coluna da direita de cada grade mostra o exemplo de treino mais próximo: o modelo não memorizou. Amostras sorteadas, não escolhidas a dedo.

Fig. 2: interpolação linear no espaço z. As transições suaves entre dígitos indicam um espaço latente contínuo.

Fortalezas, limitações

Fortalezas

- Sem cadeias de Markov: só backprop para obter gradientes.
- Qualquer função diferenciável pode ser usada em G e D.
- G recebe só gradientes vindos de D: os dados não são copiados para seus parâmetros.

Limitações

- Não há representação explícita de p_g(x).
- D e G precisam ficar sincronizados: G treinado demais leva muitos z ao mesmo x.
- Avaliação por Parzen: alta variância e fraca em alta dimensão.

Conclusões

1. As GANs introduzem uma nova abordagem para modelos generativos, baseada na competição entre um gerador G e um discriminador D.
2. O treinamento adversarial é formulado como um jogo min-max, cujo ótimo global ocorre quando p_g=p_{data} e D^*(x)=0,5.
3. A abordagem permite treinar modelos generativos utilizando retropropagação, sem necessidade de cadeias de Markov ou inferência probabilística aproximada.
4. O framework apresenta flexibilidade e vantagens computacionais, embora a estabilidade do treinamento dependa da adequada coordenação entre o gerador e o discriminador.

Quiz

Link do formulário

https://forms.gle/L84LBGhGhY3BKcYs5

Obrigado!

Inatel